

Deep Generative Models

Chapter 6: Generation by Diffusion Process

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

Diffusion Process: General SDE

General Diffusion

A generic diffusion process is given by a *stochastic differential equation (SDE)*

$$dx(t) = f(x(t), t) dt + g(t) dB(t)$$

which describes *time evolution* of location $x(t)$ for a particle impacted by the *Brownian motion* process

Note that the *Langevin equation* was a *special case* of this SDE

$$dx(t) = \nabla_x \log P(x(t)) dt + \sqrt{2} dB(t)$$

where the *drift* $f(\cdot, t)$ and *diffusion* $g(t)$ are constant in t

Diffusion Process: *Forward Process*

Let us again approximate: if we shift Δt at time t , we approximately have

$$\Delta x(t) \approx f(x(t), t) \Delta t + g(t) \Delta B(t)$$

Now, say we are at $x^{(i)} = x(t)$ and move Δt step by step; then,

$$\begin{aligned} x^{(i+1)} &\approx x^{(i)} + \Delta x(t) \\ &\approx x^{(i)} + f(x^{(i)}, t) \Delta t + g(t) [B(t + \Delta t) - B(t)] \\ &\approx x^{(i)} + f_i(x^{(i)}) \Delta t + g_i \sqrt{\Delta t} \varepsilon_i \end{aligned}$$

Here, we use the following notion

- we define $f_i(x^{(i)}) = f(x^{(i)}, t)$ and $g_i = g(t)$
- we use the fact that $B(t + \Delta t) - B(t) \sim \mathcal{N}(0, \Delta t)$

Diffusion Process: *Forward Process*

- + What is happening in this equation?
- We are simulating the **diffusion process**

This is the **forward process**

$$x^{(i+1)} = x^{(i)} + f_i(x^{(i)}) \Delta t + g_i \sqrt{\Delta t} \varepsilon_i$$

This describes a Markov chain

$$x^{(0)} \sim P_0(x) \rightarrow x^{(1)} \sim P_1(x) \rightarrow x^{(2)} \sim P_2(x) \rightarrow \dots x^{(I)} \sim P_I(x)$$

where $P_i(x)$ is roughly describing the distribution of **location** x (**t**) at time

$$t = i \Delta t$$

Reverse Diffusion Process: *Reverse SDE*

B. Anderson in 1982 showed that we can *reverse* this process *in time*

Reverse Diffusion

The reverse diffusion process is given by the following *SDE*

$$dx(s) = \left[f(x(s), s) - g(s)^2 \nabla_x \log P(\tilde{x}(s)) \right] ds + g(s) dB(s)$$

where $s = T - t$ is the *reverse time* started as T

Attention

$x(s)$ and $x(t)$ are different processes that have same distribution when $s = t$

Reverse Dynamics

- + What is really **reverse** in time?
- We can move **statistically backward** in time

We can again approximate this equation by a **reverse Markov chain**

$$x^{(i-1)} = x^{(i)} + \left[f_i \left(x^{(i)} \right) - g_i^2 \nabla_x \log P_i \left(x^{(i)} \right) \right] \Delta t + g_i \sqrt{\Delta t} \varepsilon_i$$

This describes a **backward** Markov chain: starting with $x^{(I)} \sim P_I(x)$

$$x^{(0)} \sim P_0(x) \leftarrow x^{(1)} \sim P_1(x) \leftarrow x^{(2)} \sim P_2(x) \cdots \leftarrow x^{(I)} \sim P_I(x)$$

Diffusing Back and Forth

Moral of Story

If we know a *forward diffusion* that takes us

$$\text{from } x^{(0)} \sim P^{(0)}(x) \text{ to } x^{(I)} \sim P^{(I)}(x)$$

Then, we can build the *reverse* diffusion which can take us

$$\text{back from } x^{(I)} \sim P^{(I)}(x) \text{ to } x^{(0)} \sim P^{(0)}(x)$$

- + Why should this property be helpful?!
- We know how to get from a *data sample* to *pure noise*!

Noising Diffusion: *Data to Noise*

Consider the following *forward diffusion process*

$$x^{(i+1)} = x^{(i)} - \underbrace{\beta_i x^{(i)}}_{f_i} \Delta t + \underbrace{\sqrt{2\beta_i \left(1 - \frac{\beta_i \Delta t}{2}\right)}}_{g_i} \sqrt{\Delta t} \varepsilon_i$$

If we define $\alpha^{(i)} = 1 - \beta_i \Delta t$; then, it simplifies to

$$\vec{x}^{(i+1)} = \alpha_i \vec{x}^{(i)} + \sqrt{1 - \alpha_i^2} \varepsilon_i$$

and we use the notation $\bar{\alpha}_i = \sqrt{1 - \alpha_i^2}$ for simplicity, we can write

$$x^{(i+1)} = \alpha_i x^{(i)} + \bar{\alpha}_i \varepsilon_i$$

Noising Diffusion: *Data to Noise*

The following Markov chain

$$x^{(i+1)} = \alpha_i x^{(i)} + \bar{\alpha}_i \varepsilon_i$$

describes an **Ornstein-Uhlenbeck (OU) process**: starting with a centered and normalized sample of **any distribution**

- the OU process computes always a **unit-variance** sample for next time
- as it proceeds for large I , we get

$$x^{(I)} \sim \mathcal{N}(0, 1)$$

Noising Process

We know a diffusion process that turns a **data sample** $x^{(0)}$ into **Gaussian noise**

Denoising Diffusion: Noise to Data

Since we know the **forward diffusion**: we can

build the **reverse diffusion** to get back from **latent** to **data**

This was the forward process

$$x^{(i-1)} = x^{(i)} - \hat{f}_i(x^{(i)})\Delta t + \sqrt{2\beta_i \left(1 - \frac{\beta_i\Delta t}{2}\right)} \sqrt{\Delta t} \varepsilon_i$$

where the new drift $\hat{f}_i(x^{(i)})$ is

$$\hat{f}_i(x^{(i)}) = -\beta_i x^{(i)} - 2\beta_i \left(1 - \frac{\beta_i\Delta t}{2}\right) \nabla_x \log P_i(x^{(i)})$$

Denoising Diffusion: *Latent to Data*

We can then **denoise** the **latent** into a **data sample** by

① Sample a Gaussian $x^{(I)} \sim \mathcal{N}(0, 1)$

② Proceed in reverse time as

$$x^{(i+1)} = x^{(i)} + \hat{f}_i(x^{(i)}) \Delta t + g_i(x^{(i)}) \sqrt{\Delta t} \varepsilon_i$$

③ Take $x^{(0)} \sim P(x)$ as a data sample

+ Do we have everything that we need?!

- Let's break it down

Required Scores for Denoising Diffusion

We know the diffusion process

$$x^{(0)} \sim P_{\text{data}} \rightarrow x^{(1)} \sim P_1 \rightarrow x^{(2)} \sim P_2 \rightarrow \dots \rightarrow x^{(I)} \sim \mathcal{N}(0, 1)$$

Just set a *data sample* at $\vec{x}^{(0)} \sim P_{\text{data}}$ and diffuse as

$$x^{(i+1)} = \alpha_i x^{(i)} + \bar{\alpha}_i \varepsilon_i$$

We can also sample *latent* $x^{(I)} \sim \mathcal{N}(0, 1)$, but to diffuse reversely as

$$x^{(0)} \sim P_{\text{data}} \leftarrow x^{(1)} \sim P_1 \leftarrow x^{(2)} \sim P_2 \leftarrow \dots \leftarrow x^{(I)} \sim \mathcal{N}(0, 1)$$

we need to know the following score functions

$$\nabla_x \log \mathcal{N}^0(x), \nabla_x \log P_{I-1}(x), \dots, \nabla_x \log P_1(x)$$

Score of Noisy Output

- + Back to square one! We need score function of the data!
- **No!** We need *all* scores **but** *data score!*

We need to learn these scores

$$\nabla_x \log P_{I-1}(x), \dots, \nabla_x \log P_1(x)$$

- ① $\nabla_x \log P_1(x)$ is the score of

$$x^{(1)} = \alpha^{(0)} \underbrace{x}_{\text{data sample}} + \bar{\alpha}^{(0)} \varepsilon_0$$

- ② $\nabla_x \log P_2(x)$ is the score of

$$\begin{aligned} x^{(2)} &= \alpha^{(1)} x^{(1)} + \bar{\alpha}^{(1)} \varepsilon_1 = \alpha^{(1)} \left(\alpha^{(0)} x + \bar{\alpha}^{(0)} \varepsilon_0 \right) + \bar{\alpha}^{(1)} \varepsilon_1 \\ &= \alpha^{(1)} \alpha^{(0)} x + \left(\alpha^{(1)} \bar{\alpha}^{(0)} \varepsilon_0 + \bar{\alpha}^{(1)} \varepsilon_1 \right) \end{aligned}$$

Score of Noisy Output

In fact we need the score function of a noisy version of x , i.e.,

$$z = x + \sigma \varepsilon$$

for some σ when $\varepsilon \sim \mathcal{N}(0, 1)$

- + How is it so different then?!
- It turns out to be very different! Let's take a look

Problem: Score of Noisy Sample

Let $x \sim P$ and $z = x + \sigma \varepsilon$; find the score

$$s(z) = \nabla_z \log P(z)$$

with $P(z)$ being the marginal distribution of z

Score of Noisy Output

Let's think about scalar x and z

↳ everything extends the same to multi-dimensional case

We can write

$$P(z) = \int P(z|x) P(x) dx$$

We know that $P(z|x) \equiv \mathcal{N}(x, \sigma^2)$: so, we could write

$$P(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \int \exp \left\{ -\frac{(z-x)^2}{2\sigma^2} \right\} P(x) dx$$

Score of Noisy Output

For score function we have

$$\nabla_z \log P(z) = \frac{1}{P(z)} \nabla_z P(z)$$

Considering the expression

$$P(z) = \frac{1}{\sqrt{2\pi\sigma^2}} \int \exp \left\{ -\frac{(z-x)^2}{2\sigma^2} \right\} P(x) dx$$

we can easily write

$$\begin{aligned} \nabla_z P(z) &= \frac{1}{\sqrt{2\pi\sigma^2}} \int \frac{(z-x)}{\sigma^2} \exp \left\{ -\frac{(x-z)^2}{2\sigma^2} \right\} P(x) dx \\ &= \int \frac{(x-z)}{\sigma^2} P(z|x) P(x) dx \end{aligned}$$

Score of Noisy Output

So, we can write

$$\begin{aligned}
 \nabla_z \log P(z) &= \frac{1}{P(z)} \int \frac{(x - z)}{\sigma^2} P(z|x) P(x) dx \\
 &= \frac{1}{P(z)} \int \frac{(x - z)}{\sigma^2} P(z, x) dx \\
 &= \frac{1}{P(z)} \int \frac{(x - z)}{\sigma^2} P(x|z) P(z) dx \\
 &= \int \frac{(x - z)}{\sigma^2} P(x|z) dx = \mathbb{E}_{x \sim P|z} \left\{ \frac{(x - z)}{\sigma^2} \right\}
 \end{aligned}$$

This concludes that

$$s(z) = \frac{1}{\sigma^2} \mathbb{E}_x \{x - z|z\} = \frac{1}{\sigma^2} \left(\underbrace{\mathbb{E}_x \{x|z\}}_{\hat{x}(z)} - z \right) = \frac{\hat{x}(z) - z}{\sigma^2}$$

Recap: Estimating from Noisy Observation

Before we start with this section: let us start with a simple problem

Say we have data x and we see a noisy version of it as

$$z = x + \sigma \epsilon$$

for noise process $\epsilon \sim \mathcal{N}^0$

We intend to find the optimal function that

takes z and *denoises* it to an estimate of x

Bayesian Optimal Estimator

Let's think scalar

$$\hat{x}(z) = \operatorname{argmin}_u \mathbb{E} \left\{ (u - x)^2 | z \right\}$$

To find the optimizer, we can write

$$\frac{d}{du} \mathbb{E} \left\{ (u - x)^2 | z \right\} = 0$$

This concludes

$$\mathbb{E} \left\{ (u - x) | z \right\} = 0 \rightsquigarrow u^* = \mathbb{E} \{ x | z \}$$

Bayes Optimal Denoiser

The optimal estimate in the Bayesian sense is

$$\hat{x}(z) = \mathbb{E} \{ x | z \}$$

Bayesian Denoiser

- + Can we estimate the Bayes optimal denoiser by *sampling*?
- Not really!

To estimate Bayes optimal denoiser by sampling: we need to

- sample many data samples x whose noisy version is

exactly z

- use these samples to compute the estimator as

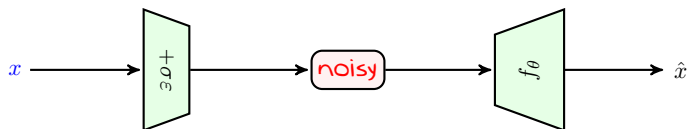
$$\hat{x}(z) = \mathbb{E}\{x|z\}$$

Complexity of Bayes Optimal Denoiser

To estimate the Bayes optimal denoiser, we need to collect *exponentially large data samples*, such that we have enough samples for each z !

Computational Denoising: *Decoding*

We can though train a computational denoiser



We train this model on a dataset using risk

$$\begin{aligned}
 R(\theta) &= \mathbb{E}_x \{ \|\hat{x} - x\|^2 \} \\
 &= \mathbb{E}_x \{ \|f_\theta(x + \sigma\varepsilon) - x\|^2 \}
 \end{aligned}$$

This is equivalent to autoencoding with *noisy encoder* and decoder f_θ

Score of Noisy Output

We could also say that

$$z = x + \sigma \epsilon \rightsquigarrow \epsilon = \frac{z - x}{\sigma}$$

So, we could also write

$$s(z) = \frac{1}{\sigma} \mathbb{E}_x \left\{ \frac{x - z}{\sigma} \middle| z \right\} = -\frac{1}{\sigma} \mathbb{E} \{ \epsilon | z \}$$

Score of Noisy Output

Problem: Score of Noisy Sample

Let $x \sim P$ and $z = x + \sigma \varepsilon \rightsquigarrow$ find the score $s(z)$

The score at point z is given by

- either the conditional noise average

$$s(z) = -\frac{\mathbb{E}\{\varepsilon|z\}}{\sigma}$$

- or the Bayes optimal denoiser as

$$s(z) = \frac{\hat{x}(z) - z}{\sigma^2}$$

Computing Score by Noise Averaging

Say we need to find score $s(z)$ while we have access to a bunch of noise samples ϵ^j with the following property

↳ All $z - \sigma\epsilon$ end up with a *data sample*

We can then estimate the score as

$$s(z) = -\frac{\mathbb{E}\{\epsilon|z\}}{\sigma} \approx -\frac{\hat{\mathbb{E}}\{\epsilon|z\}}{\sigma}$$

- + But how could we *a bunch* of such noise samples?!
- We rely on the *single* sample that we have 😊

In practice we estimate as

$$s(z) \approx -\frac{\epsilon}{\sigma}$$

Computing Score by Denoising

Say we need to find score $s(z)$ while we have access to data sample x : we take x and sample $\varepsilon \sim \mathcal{N}^0$; then, we **noise** the data sample as

$$z = x + \sigma \varepsilon$$

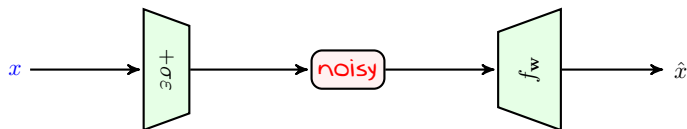
We can then use the denoiser $\hat{x}(z)$ to compute the score as

$$s(z) = \frac{\hat{x}(z) - z}{\sigma^2}$$

- + But how could we get the denoiser $\hat{x}(z)$?!
 - We can learn it!

Computational Approach: *Decoding*

We saw before that we can *denoise* by a computational model



We just need to train this model by risk

$$\begin{aligned}
 R(\mathbf{w}) &= \mathbb{E}_{\mathbf{x}} \left\{ \|\hat{x} - \mathbf{x}\|^2 \right\} \\
 &= \mathbb{E}_{\mathbf{x}} \left\{ \|f_{\mathbf{w}}(\mathbf{x} + \sigma \varepsilon) - \mathbf{x}\|^2 \right\}
 \end{aligned}$$

Training Diffusion Score Model: *Direct Score Model*

We consider a *score model* $s_{\mathbf{w}} : \mathbb{R}^d \times \mathbb{R} \mapsto \mathbb{R}^d$ which

takes a *noisy* sample z and *noise standard deviation* $\sigma \rightsquigarrow$ returns $s(z)$

To train this model we make data batches as follows

For $j = 1, \dots, n$

- ① Sample a *data sample* x^j and choose a value for σ^j
- ② Noise data sample as $z^j = x^j + \sigma^j \varepsilon^j$ for some $\varepsilon^j \sim \mathcal{N}^0$
- ③ Set the data point and its *label (score)* to

$$\text{input to } s_{\mathbf{w}} \equiv (z^j, \sigma^j) \quad \text{output of } s_{\mathbf{w}} \equiv -\varepsilon^j / \sigma^j$$

We could train this network by minimizing the empirical loss

$$\hat{R}(\mathbf{w}) = \hat{\mathbb{E}} \{ \|s_{\mathbf{w}}(z^j, \sigma^j) + \varepsilon^j / \sigma^j\|^2 \}$$

Training Diffusion Score Model: Denoising Model

We consider a **denoising model** $f_{\theta} : \mathbb{R}^d \times \mathbb{R} \mapsto \mathbb{R}^d$ which

takes a **noisy** sample z and **noise standard deviation** $\sigma \rightsquigarrow$ **denoises** to $\hat{x}(z)$

To train this model we make data batches as follows

For $j = 1, \dots, n$

- ① Sample a **data sample** x^j and choose a value for σ^j
- ② Noise data sample as $z^j = x^j + \sigma^j \varepsilon^j$ for some $\varepsilon^j \sim \mathcal{N}^0$
- ③ Set the data point and its **label (recovery)** to

input to $f_w \equiv (z^j, \sigma^j)$ output of $f_w \equiv x^j$

We can use this denoiser to estimate the score as

$$s_w(z, \sigma) = \frac{f_w(z, \sigma) - z}{\sigma^2}$$

Sampling Diffusion Score Model: *Forward Diffusion*

We define a **forward diffusion** as

$$x^{(i+1)} = \alpha_i x^{(i)} + \bar{\alpha}_i \varepsilon_i$$

- + How should we choose α_i ?!
 - Well! For sure $\bar{\alpha}_i$ *is small*, since we defined $\alpha_i = 1 - \beta_i \Delta t$

For this process we can say

$$\begin{aligned}
 x^{(1)} &= \alpha_0 x + \bar{\alpha}_0 \varepsilon_0 &= \theta_0 x + \sigma_0 \varepsilon^{(0)} \\
 x^{(2)} &= \alpha_1 \alpha_0 x + (\alpha_1 \bar{\alpha}_0 \varepsilon_0 + \bar{\alpha}_1 \varepsilon_1) &= \theta^{(1)} x + \sigma_1 \varepsilon^{(1)} \\
 &\vdots
 \end{aligned}$$

with all $\varepsilon_i \sim \mathcal{N}^0$

Sampling Diffusion Score Model: *Reverse Diffusion*

From the **forward diffusion**: we can build the **reverse** diffusion as

$$x^{(i-1)} = \left[f_i x^{(i)} - g_i^2 s \left(x^{(i)} \right) \right] \Delta t + g_i \sqrt{\Delta t} \varepsilon_i$$

Noting that

$$x^{(i)} = \theta_{i-1} x + \sigma_{i-1} \varepsilon^{(i-1)}$$

We use $s_{\mathbf{w}}$ to estimate the score as

$$s \left(x^{(i)} \right) \approx s_{\mathbf{w}} \left(x^{(i)}, \sigma_{i-1} \right)$$

We keep going **backward** till we get to

$$x^{(0)} \sim P_0 \equiv P_{\text{data}}$$